Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Yusheng Zheng
UC Santa Cruz
yzhen165@ucsc.edu

Abstract

Re-checking a formal proof is usually far cheaper than finding it, because a proof checker validates a stored proof term and never repeats the search that produced it. This assumption fails for proofs by computation, whose terms ask the kernel to evaluate a decision procedure: every check repeats the evaluation. We measure this effect on OpenAI's AI-generated Lean library of 163,096 modules. Re-checking two computational certificates takes 80–82% of their compilation time, against 0.5–7% for ordinary proofs. Such certificates are common in AI-generated Lean: the library uses `decide +kernel` 10,668 times more densely than Mathlib. Compiling 162,170 of its modules took at least 169 P-core and 302 E-core CPU-hours, a quarter of which went to 1% of the modules, and a default build skips a fifth of the source files, which hold a third of the computational proofs. Lean's tooling adds fixed costs that can exceed proof checking itself: import loading, module discovery and the `-M` memory guard. We release ProofServe, a tool that re-checks Lean projects in parallel with Lean's own kernel and reports what each check covered; eight workers shorten wall time by 1.7–2.1×.

1 Introduction

AI systems now write formal mathematics at the scale of whole libraries. OpenAI's public mathematics repository [17] backs its results with 163,096 Lean modules at the revision we study, about nineteen times as many files as the Mathlib snapshot it builds on. Before such a library is reused, cited or merged, someone has to re-check it independently of the system that produced it. The common expectation is that this step is cheap. Lean separates proof search from proof checking: tactics may search for minutes, but the kernel validates only the resulting proof term, and an independent checker replays stored terms without any search [3, 11].

This expectation holds for ordinary mathematical proofs and fails for proofs by computation. A proof produced by `decide` states that a decision procedure evaluates to `true`, and the kernel can accept it only by running that evaluation. Re-checking such a proof recomputes it. An independent report on the same library completed a full build and suggested kernel computation as the reason some checks are expensive, based on three source examples [16]. We measure this explanation directly and ask what it means for anyone who has to re-check an AI-generated library. We make four contributions.

- **Two cost regimes.** Re-checking ordinary proofs costs 0.5–7% of their compilation time, while computational certificates cost 80–82%. Across 31 modules the regimes blur into a spectrum, and project-wide averages hide the expensive certificates (§4.1).
- **A per-module census of an AI-generated library.** We compiled 162,170 of 163,096 modules and recorded the cost of each. The library relies on computation far more than Mathlib, its cost is concentrated in a small tail that file size does not predict well, and its default build skips a third of its computational proofs (§4.2–4.4).
- **Costs in Lean's tooling.** Importing dependencies, discovering modules and Lean's `-M` memory guard can each cost more than the proof checking they surround. Minimal examples show what Lean's checker re-checks in its two modes (§5).
- **ProofServe**, an open-source tool that re-checks a compiled Lean project with the original kernel in parallel and reports coverage and trust assumptions (§6).

2 Background

Elaboration and kernel checking. Lean compiles a module in two stages. The elaborator runs tactics such as `simp`, `omega` and `linarith`, which may search, backtrack and call decision procedures, and produces a proof term that records only the successful steps. The kernel then type-checks the term, and Lean stores the checked declarations in an `.olean` file. Type checking includes unfolding definitions and reducing terms, so a short term can require an arbitrary amount of computation to check.

Proof by computation. For a decidable proposition p, `decide` produces a proof whose validity requires the kernel to reduce `decide` p to `true` [8]. Plain `decide` evaluates the instance twice during compilation, once in the elaborator and once in the kernel; `decide +kernel` skips the elaborator's evaluation. In both forms the evaluation is part of the kernel check, so every re-check repeats it. `decide`

`+native` instead runs compiled code and admits the result through an axiom, which enlarges the trusted base.

Re-checking and coverage. Lean ships `leanchecker`, which loads stored declarations and replays them through the kernel without running tactics [7, 11]. Its default mode trusts imported modules and checks only the declarations that the target module adds; `-fresh` replays the whole imported environment into an empty kernel environment. Lean 4.35 adds `lake check`, which re-checks a package's default targets [10]. A normal build and `lake check` both start from the package's declared root targets, so files that no root imports are neither compiled nor checked.

3 Methodology

Corpus and machine. We pin the OpenAI repository at commit `4eaee875`: 163,096 Lean modules plus one build configuration, on Lean 4.34.1 and Mathlib `d13f23b7` [17, 4]. All measurements run on one Intel Core i9-14900K with 8 performance (P) cores, 16 efficiency (E) cores, no SMT, 64 GB of memory and Linux 7.0. P and E cores do different amounts of work per second, so we report their CPU time separately and never convert one into the other. Workloads of other users kept running on the machine; our timings include that interference. Except for the import measurements in §5.1, no run uses Lean's `-M` option; memory limits come from cgroups (§5.3 explains why).

Checking versus compiling. For four modules we time reading and kernel-checking one module after its dependencies are loaded, and divide by the wall time of compiling that module from source. For 31 further modules we compare the CPU time of complete checker and compiler processes. These runs use the eight P cores, one task at a time, with warm file caches.

Library census. We compiled the library in dependency order, one process per module with Lean's standard `-j8` setting, running up to 22 modules at once on the 8 P cores and 14 E cores. For every module we record CPU and wall time, peak resident memory, and hardware instruction counts. Families of modules that repeatedly exhausted memory were deferred together with their dependents instead of being retried indefinitely.

Comparing modules across cores. Modules ran on different core types and under different load, so we compare them by instruction counts. In a calibration of 240 compilations under four conditions (P or E cores, quiet or loaded), the user-mode instruction count of the same module varied by a median of 0.018% (95th percentile 0.14%), while kernel-mode counts varied by a median of 8.6% and repeated CPU time by up to 20% (95th percentile). We therefore compare modules by user-mode instructions. We chose this metric after seeing the variation: total instruction counts missed our preset targets of 1% median and 3% 95th-percentile variation.

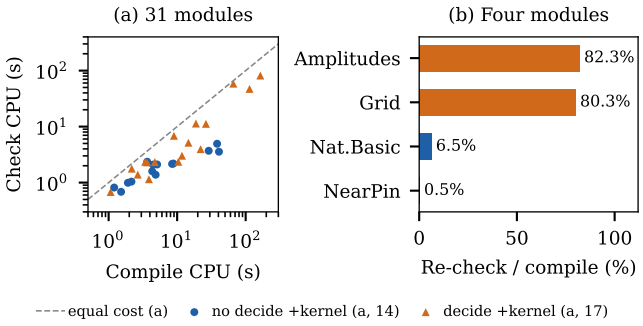


Figure 1: Checking cost relative to compilation. (a) CPU time of complete checker and compiler processes for 31 modules, by whether the source contains `decide +kernel`; the two groups overlap. (b) Time to re-check one module after its dependencies are loaded, as a share of the module's compilation wall time: ordinary proofs (blue) and computational certificates (orange).

Lexical scan and profiles. We count `decide +kernel` calls after masking comments and strings; the count marks where computation is likely and does not measure its cost. To study Lean's tooling we use `perf`, system-call tracing, and minimal examples run with the official Lean 4.34.1 binaries.

4 Checking and compiling the library

4.1 Re-checking repeats the computation

Figure 1(b) shows the central measurement. Two computational certificates from the AI library, `Grid` and `Amplitudes`, re-check in 80.3% and 82.4% of their compilation wall time. Their re-checks take 255.5 and 110.0 seconds, almost all of it inside the kernel's replay call. Two ordinary modules, Mathlib's `Nat.Basic` and the AI library's `NearPin`, re-check in 6.5% and 0.5% of their compilation time.

The difference follows from what each proof term stores. A tactic that searches for a derivation emits a term recording the steps that worked, and the kernel validates those steps without redoing the search. A certificate checked through `decide` stores a compact claim that a decision procedure returns `true` on a large finite object, and the kernel validates it by running the procedure. The compilation cost of a certificate is mostly that evaluation, so a re-check inherits it. `Grid` is a typical case. It certifies rational enclosures of quadrature points and their Fourier kernels in 590 lemmas with 1,664 `decide +kernel` calls, most of them shaped like this abridged excerpt:

```
def p0_0 : PointBox :=
  <<166665097940216761888082528704,
    166665097940216761888082539539>, ...>
lemma p0_0_contains :
    p0_0.Contains (quadraturePoint 0 0) := by
  ...
  have he : PointBox.calc p0_0.t = p0_0 := by
    decide +kernel
```

The proof is one line, but checking it makes the kernel evaluate `PointBox.calc` on 30-digit rationals and compare the result with the stored box. Every re-check repeats that arithmetic.

The two regimes are ends of a spectrum rather than two classes (Figure 1(a)). Across 31 modules, the checker used a median 45% of the compiler's CPU time: 50% for the 17 modules containing `decide +kernel` and 39% for the 14 without, with overlapping distributions. Plain `decide` and other tactics can also leave large evaluations in a term. A Ramsey K_{10} certificate with no `decide +kernel` in its source needed 356 CPU-seconds to check against 1,725 to compile (21%).

Averages over a project hide these modules. Checking the full dependency closure of the `Falconer` development cost 6.3% of its compilation CPU, and a Mathlib closure 9.0%, because many cheap dependencies dilute a few expensive certificates. Planning a re-check therefore needs per-module costs: a project-wide ratio says little about how long one certificate takes, and a short source file can be among the most expensive to check.

4.2 AI-generated Lean relies on computation

After masking comments and strings, the AI library contains 1,354,379 `decide +kernel` calls in 5,848 files. The Mathlib snapshot it builds on contains seven, in four of its 8,529 files. Normalized by source size, the AI library uses this construct 10,668 times more densely. We did not test why. A likely factor is that many AI-generated results concern specific finite objects, where exhaustive evaluation is a proof that search finds reliably, while Mathlib develops general theories.

4.3 Compiling the library

Of 163,096 modules, 162,170 (99.43%) compiled and none ended in failure. We deferred the remaining 926 because a few families kept exhausting the memory available to their build jobs; we set those families aside together with every module that depends on them. They include `PadicHodge`, `BigCrystalline` and `Ramsey.CycleClique.Construction`. The completed part took 31.5 hours of wall time, 169.0 P-core CPU-hours and 302.4 E-core CPU-hours. These are lower bounds for the whole library: the deferred modules are the most memory-hungry ones, and dependency order is not a random sample, so we do not extrapolate.

The cost is concentrated (Figure 2a). The median module needed 4.0 CPU-seconds on a P core and 7.5 on an E core, or 1.3×10^{10} user-mode instructions, while the costliest 1% of modules used a quarter of the CPU time on each core type. The 5,848 files that contain `decide +kernel` are 3.6% of the modules, account for 18% of the CPU time and make up 41–45% of the costliest 1%.

Source size is only a moderate predictor of cost (Figure 2b; Spearman rank correlation 0.60–0.62). Files under 1 KiB still took a median of 5.8 E-core seconds, partly

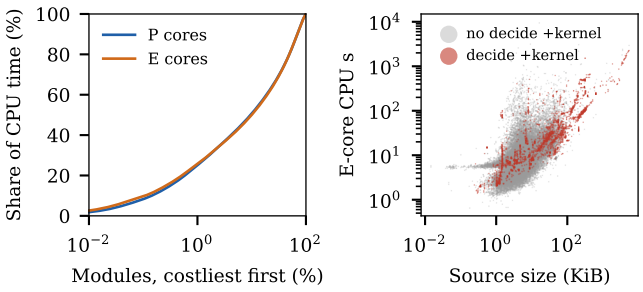


Figure 2: Compilation cost across 162,170 modules. (a) Cumulative share of CPU time against the share of modules, costliest first; P and E cores are shown separately. (b) Source size against E-core CPU time; modules that contain `decide +kernel` are red.

the fixed cost of loading imports (§5.1). At the other end, the single costliest module, `OAI.Analysis.FreeWords.UltraExpectationFull`, is a 15.5 KiB file with no `decide +kernel`; it took 9,314 E-core CPU-seconds, while a 5.0 MiB Ramsey certificate file with 11,114 `decide +kernel` calls took 2,241. Among the costliest 1% of modules, 10–17% are smaller than the median file.

Memory also limited parallelism. The median compile process reached a peak resident size of 6.2–6.4 GiB and the largest 25 GiB. Resident size counts memory-mapped `.olean` pages that concurrent processes can share, so it overstates exclusive use, yet 65 compile attempts were still stopped by memory limits; their modules succeeded on retry or belong to the deferred families.

The earlier report on this library built 122,458 modules of an older revision in 13.8 hours on 32 cloud vCPUs, and estimated about 410 CPU-hours, including roughly 330 hours of compilation extrapolated from 26 timed modules [16]. Different revisions, hardware and scope prevent a direct comparison; the census replaces that extrapolation with a measured cost for each module.

4.4 A default build covers four fifths of the library

A successful default build is the usual evidence that a Lean project checks. In this library, the declared roots reach 130,834 of 163,097 source units. The other 32,263 files (19.8%) are never compiled by a default build, and they contain 451,008 of the 1,354,379 `decide +kernel` calls (33.3%). The earlier report found 30,697 files outside the default graph at its revision, about a quarter of its inventory [16]. A claim that such a library "checks" should state which roots were built and how many files they reach.

5 What Lean's tooling costs

Re-checking a library involves more than kernel checking. Lean's tooling adds fixed costs around it, and each of the three below can exceed the cost of checking an ordinary proof (Table 1).

Operation	Setting compared	Fast	Slow
Check `Nat.Basic`	search path	0.39	322.9
Compile `Grid`	memory limit	306	622
Compile `UMoments9`	memory limit	88	142
Compile `EnvelopePsiB`	memory limit	63.5	64.4

Table 1: Tooling costs, wall seconds. Search path: only the root's 1,330 imported modules versus the full 90-entry build path (one run each). Memory limit: a cgroup versus Lean's `-M` (medians of three). Verdicts and outputs are identical in each pair.

5.1 Loading imports

Every Lean process starts by loading the modules it imports. For 13 short modules whose builds took under five seconds, importing their dependencies and doing nothing else took a median of 1.36 seconds (0.71–3.20) and 68% of their build time, and peaked at 3.1 GiB resident, 93% of it file-backed `.olean` mappings. On these modules the official checker took a median 1.28 times as long as the build: re-checking a short module costs more than compiling it, because both pay for imports and the checker adds module discovery. A profile of a process that only imports Mathlib attributes the time to marking imported objects persistent (`lean_mark_persistent`, 9.3% of samples), name comparison (`lean_name_eq`, 5.7%), environment finalization and hash-table insertion, memory allocation (about 6%) and file-path lookup in the kernel. These runs used `-M`, so their system time may include the accounting cost described in §5.3. For a library checked one module per process, this fixed cost dominates the ordinary proofs, and sharing it across modules is the first thing a library-scale checker needs. Verification servers for AI provers face the same cost per request, which is why Kimina's Lean server keeps imported environments loaded between requests [18].

5.2 Discovering modules

Before it checks anything, `leanchecker` enumerates every `.olean` file on the search path, resolves each file name to a module name, and only then selects the requested root [7]. In the AI library's build environment, whose search path has 90 entries and yields 203,273 `.olean` path entries, checking Mathlib's `Mathlib.Data.Nat.Basic` in the default mode took 322.9 seconds, 234.6 of them in system CPU. With a search path containing only the 1,330 modules that this root imports, the same check took 0.39 seconds. A probe that runs only the discovery steps spent 2.6 seconds enumerating directories and 271.4 seconds resolving module names. These are single observations in an unusually large environment; the practical remedy is a search path limited to the closure being checked.

5.3 The `-M` memory guard

Lean's `-M` option sets a memory budget [9]. With it, every 200th call to Lean's memory check invokes `getrusage`, which sums resource use over all threads of the process [14], and the kernel runs this check throughout reduction. Compiling `Grid` with eight threads made 4.0 million `getrusage` calls; resource accounting took 75% of the sampled kernel-mode cycles and futex synchronization another 20%. Removing `-M` while enforcing the same 15 GiB limit through a cgroup [13] cut `Grid`'s compile time from 622 to 306 seconds and its system time from 436 to 107 seconds (medians of three runs). Another certificate, `UMoments9`, went from 142 to 88 seconds, while an ordinary module, `EnvelopePsiB`, barely changed (64.4 to 63.5 seconds). All 210 output files were byte-identical. For computation-heavy proofs, a limit enforced outside the Lean process gives the same protection without the accounting cost.

5.4 What the two checker modes re-check

Minimal examples run through the official `leanchecker` show that its two modes can give different verdicts on the same artifact. After we removed a module's `.olean.private` part, the default mode accepted the module and `-fresh` failed to load it. A proof using the deprecated native-evaluation bridge `Lean.ofReduceBool` passed the default mode and failed under `-fresh`, which replays declarations into an empty environment without the compiled code that the evaluation needs.

Checking the merged environment also differs from checking every module. When two modules prove a theorem with the same name and statement, importing both keeps one body, the one imported last. We corrupted that body in one module by editing its serialized file. Checked as its own root, the corrupted module was rejected by both modes. A root importing the good module first and the corrupted one second was rejected by `-fresh` and accepted by the default mode, which does not re-check imports. With the import order reversed, the good body replaced the corrupted one and both modes accepted. None of these cases involves normally compiled source proving a false statement; they show that verifying every module's proofs requires checking each module in its own declared import context.

6 ProofServe

ProofServe turns these measurements into a re-checking workflow with two commands. `proofserve check` takes a compiled project, finds its roots and import closure, and reports missing artifacts. Within the closure it builds a graph of declarations from the constants they reference and checks ready declarations in parallel waves, each through Lean's original kernel against the already-checked frontier. Between waves it marks newly checked objects persistent so workers can share them. Declarations other than ordinary definitions and theorems, such as inductive types, go through Lean's own replay path first. Work is divided into checkpointed segments, so an interrupted check resumes from the last completed segment. A receipt records the Lean and dependency versions, the module set, accepted and rejected modules, the axioms

Closure	Clock	1	2	4	8
`Nat.Basic`	CPU	110.3	111.3	112.1	113.5
1,330 modules	Wall	110.8	82.7	69.7	65.3
`Grid`	CPU	751.2	778.6	799.1	835.8
3,793 modules	Wall	754.0	517.3	398.4	352.4
`Barrier`	CPU	732.6	–	–	766.3
4,449 modules	Wall	735.6	–	–	425.9

Table 2: ProofServe check time in seconds by number of workers (median of three fresh runs, default mode). Barrier was run at one and eight workers only.

they declare, and CPU and wall time. `proofserve audit` compares the source inventory, the default import graph and a receipt: it reports which files and `decide +kernel` calls the default roots miss and what the receipt actually covered.

Trust. ProofServe trusts Lean's kernel, its artifact reader, the coordinator and local checkpoints. By default it reuses a kernel-checked theorem body when another module declares a theorem with the same interface. This saves repeated work but, as in §5.4, it can accept a crafted artifact that official checking rejects in its declared import context. `-strict-theorem-bodies` checks each body in its own context.

Evaluation. Table 2 reports the release build on three closures. Eight workers shorten wall time by 1.70, 2.14 and 1.73 times for `Nat.Basic`, `Grid` and `Barrier`, while total CPU rises by 2.9, 11.3 and 4.6%. Strict mode costs 4.23 times the wall time and 2.50 times the CPU on `Nat.Basic` at eight workers; on `Grid` it timed out at 1,800 seconds, at least 5.1 times the default. Turning off persistent marking adds 12–17% CPU. A deliberately interrupted check resumed with 17% extra wall time and the same result.

We also checked three external projects: Newspeak, Solar, and `samuelburbulla/proofs`, a collection its authors describe as AI-generated. ProofServe and the official checker accepted all 126 roots they have in common. The two did different amounts of work: ProofServe checked the full closures of 754, 2,368 and 10,753 modules in 109, 167 and 2,481 CPU-seconds, while the official default mode checked only the projects' own 105, 12 and 9 root modules, trusting their imports, in 38.6, 5.5 and 37.4 CPU-seconds. The numbers are therefore not a speedup comparison.

7 Implications

Our measurements suggest five practices for re-checking AI-generated libraries. First, budget per certificate: a computational proof costs close to its compilation time, and project averages understate it. Second, measure how much of a library relies on computation before sizing the job; a lexical scan for `decide +kernel` found the 3.6% of modules that used 18% of the compilation CPU. Third, report coverage: name the roots that were built or checked and how many source files they reach, since a default build missed a fifth of this library. Fourth, amortize fixed costs: load imports once per worker rather than once per module, and give the checker a search path limited to the closure it checks. Fifth, enforce memory limits outside Lean, with cgroups or resource limits, rather than with `-M`.

8 Limitations

All results come from one machine, one revision of one library and Lean 4.34.1. The checking-versus-compiling comparison uses a few selected modules, and the 31-module panel shows a spectrum rather than two clean classes. `decide +kernel` counts are lexical proxies. The census is a lower bound over a dependency-ordered subset that excludes the most memory-hungry families, and we chose its comparison metric after observing measurement variation. The tooling measurements in §5.2 are single observations. A complete re-check of the library with ProofServe did not finish within our time budget: it certified 968 of the 162,170 compiled modules. The coordinator has not been formally verified or independently audited, receipts are not bound to a source digest, and checkpoints are trusted local state rather than attested evidence.

9 Related work

Independent re-checking has a long history in Lean: `lean4checker`, the bundled `leanchecker`, external kernels such as nanoda [15] and Lean4Lean [2], and `lake check` [11, 10], following the Lean FRO's agenda for external checking [12]. Parallel and asynchronous proof checking appears in Isabelle [19], Coq [1] and Lean's parallel elaboration and kernel [6, 5]. Kimina and AXLE provide verification services for AI provers [18, 20]. The closest work is the report on the same library, which built it in the cloud, ran extensive independent checking, measured nanoda with four threads at a median 0.21 of Lean's single-threaded replay time, and proposed kernel computation as the cause of expensive checks [16]. We contribute matched measurements of that cause, a per-module census, measurements of Lean's tooling costs, and a usable original-kernel tool, rather than new parallelism or serving mechanisms.

10 Conclusion

Checking a proof is cheap only when the proof is a record of search. When the proof is a computation, checking repeats it, and AI-generated Lean uses such proofs far more than human-written Mathlib. At library scale, the fixed costs of Lean's tooling and the coverage of a default build matter as much as the kernel itself. ProofServe, the census data and every script behind these numbers are available at `https://github.com/yunwei37/proofserve` (release v0.1.0; checker `e000ed62`).

References

[1] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[2] Mario Carneiro. Lean4lean: Verifying a typechecker for lean, in lean, 2025. Version 3, revised 14 September 2025; kernel replay, not source-compilation timing.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[4] Lean community. Mathlib4 reference source at d13f23b7. https://github.com/leanprover-community/mathlib4/tree/d13f23b723b8a846827a245b89c10fc7d3f11612/Mathlib, 2026. Pinned Mathlib subtree; local Git revision and unchanged source checked 1 November 2026. Lexical census excludes comments and strings.

[5] Lean contributors. Lean 4.17.0: Parallel kernel checking, 2025. Kernel checking concurrent with elaboration.

[6] Lean contributors. Lean 4.19.0: Parallel elaboration, 2025.

[7] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026.

[8] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026.

[9] Lean contributors. Lean 4.34.1 runtime memory-budget checks, 2026. Source and matching SDK disassembly inspected 31 October 2026.

[10] Lean contributors. Lean 4.35.0 release notes, 2026.

[11] Lean contributors. lean4checker and the built-in leanchecker, 2026.

[12] Lean FRO contributors. The lean fro year 4, part 1 roadmap, 2026.

[13] Linux kernel contributors. Control group v2: memory controller, 2026. Authoritative kernel documentation for memory.max, accounting and OOM semantics; accessed 31 October 2026.

[14] Linux man-pages contributors. getrusage(2): process resource-usage semantics, 2026. RUSAGE SELF aggregates usage across process threads; accessed 31 October 2026.

[15] Nanoda contributors. Nanoda: external lean type checker, 2026. Independent kernel implementation.

[16] nealmcb. Verifying openai/math in lean: cost and checker study. Technical report, revision c9570a82, 2026. Full report inspected 3 November 2026.

[17] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[18] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[19] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009.

[20] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.